

# Module TS Supports Legacy Integration

- Document number: P0909R0
- Date: 2018-02-11
- Author: Steve Downey <sdowney2@bloomberg.net>
- Audience: LEWG, Core

## Abstract

We believe the model for modules as proposed is fundamentally sound. With the changes published in N4720 it is now minimally possible to convert a very large codebase to use modules in a purely additive manner. Existing code will not require rewrite in order to be exposed through a module interface. A new module interface unit can be written that exports names without requiring that the existing source code be modified. Current consumers of a library can continue to use the existing library and headers without requiring that all are migrated to modules at once.

Preprocessor macros are not part of the model for the Modules TS. This will mean there are some libraries that can not be modularized without changes. However, with the wording in earlier versions of the TS, it would not have been feasible for any library.

## Core Use Case

Bloomberg has a very large existing base of C++ code. We do not believe we are unique, and we do believe our concerns are shared by any entity with a large existing investment in C++.

## Background

1. Over 250,000 translation units
2. Atomic changes across the codebase are not possible
3. The same code is built on 4 major operating systems using 4 different compilers

## Business Requirements

1. Core services must always build
2. Breaking non-core services is discouraged
3. Both bug fixes and features are delivered at tip of tree, not to old versions

This has several implications for creating a module for an existing facility. The pre-existing header must still work. It must be possible to use both the existing header and the module in one program. It must also be possible to use the header and the module in a single translation unit, mentioning them in either order, as transitive includes are converted to imports.

## Solution as outlined in N4270

From [dcl.module.interface] in N4270, in paragraph 2:

If the declaration is a using-declaration (10.3.3), any entity to which the using-declarator ultimately refers shall have been introduced with a name having external linkage. [Example:  
`int f()` // `f` has external linkage

```
export module M;
export using ::f; // OK
```

In the example, the name `f` from the global namespace is exported from the module with the same semantic properties it had at the point it was exported. For names attached to entities with module linkage this can introduce subtle issues; however names which are made available by inclusion into the module interface unit from an existing header will be exactly as usable to an importing translation unit as to one that included it. They will have the same semantic properties.

The exported using declaration will also have the same associated entities as the original definition. This may allow a tighter control of names made visible to clients through the module interface than through the header file. It may be possible, for example, to choose to not export names from a detail namespace. Those entities will be reachable for use, but not directly usable by clients.

## Example

```
// facility.h
#include <other_library.h>

namespace facility {
class Foo {};
int func(Foo const& foo);
int func(Foo&& foo);
enum BAR { ONE, TWO, THREE };
// ...
} // namespace facility
```

```
// facility module interface unit
#include <facility.h> // outside the purview of module facility

export module facility;

export namespace facility {}
export using facility::Foo; // exports complete type Foo
export using facility::func; // exports both func from facility
export using facility::BAR; // exports BAR, and associated enumerators
```

```
// main.cpp
import facility;

int main() {
    facility::Foo f;
    int i = facility::func(f);
    facility::BAR b = facility::ONE;
    return i;
}
```

The `facility.h` header is included in the module interface outside the purview of the module. This ensures that none of the entities declared in `facility.h` have module linkage. This maintains compatibility with the existing library where the entities from the facility are defined. This is critical for interoperability with translation units using the module interface and translation units using the header. It is important, also, that it is not an error to both import the module and to include the header. The names made available via the module are the same as those declared in the header.

The header `other_library.h` is not exported at all. However, if any of the names exported from the module have associated entities provided by that header, those will be made available to the importer of the module. This is important, as it bounds the domain the implementor of the module interface unit must consider.

## Conclusion

The published Modules TS now supports the minimum capabilities necessary for safely modularizing an existing library in an additive manor. Only new code in the module interface unit needs to be written in order for clients to consume the library via `import`.

Libraries that publish macros to clients, such as logging facilities, which typically need to reliably capture file and line information at the invocation site, are still an issue.